

Работа № 4. Физическая организация данных и индексация поиска

Общие сведения о работе

Цель работы – получение навыков оценки влияния организации данных и метода доступа при поиске на производительность процессов выборки и модификации информации в базе данных.

Индексация данных

Индекс представляет собой отдельную физическую структуру данных, которая позволяет получать быстрый доступ к одной или нескольким строкам данных. Таким образом, правильная настройка индексов является ключевым аспектом улучшения производительности запросов. Внутри СУБД индекс представляет собой, как правило, B^+ дерево или Хэш-структуру, построенную, в основном, по принципам расширенного хеширования данных.

За счет ускорения доступа к данным, индексы позволяют повысить эффективность операций выборки данных, однако при каждом изменении данных, связанных индексом, изменения также должны быть внесены и в сам индекс – что замедляет операции обновления, вставки и удаления данных (операции модификации).

В общем случае, выполняя запрос, оптимизатор СУБД может как воспользоваться индексом, так и проигнорировать его. Как правило, вторая ситуация встречается, когда в диапазон выборки попадает более половины всех кортежей и поиск их по индексным структурам теряет смысл. В СУБД можно выделить два типа индексов.

Кластеризованный индекс определяет физический порядок данных в таблице. Можно создавать для таблицы лишь один кластеризованный индекс, т. к. строки таблицы нельзя упорядочить физически более чем одним способом.

Поиск с использованием кластеризованного индекса выполняется от корневого узла B^+ -дерева по направлению к листьям дерева, которые связаны между собой в двунаправленный связанный список, называемый цепочкой страниц.

Важным свойством кластеризованного индекса является то, что его листья дерева (узлы-листья) содержат страницы данных. (Узлы кластеризованного индекса всех других уровней содержат страницы индекса.) Таблица, для которой определен кластеризованный индекс (явно или неявно), называется кластеризованной таблицей.

Кластеризованный индекс создается по умолчанию для каждой таблицы, для которой с помощью ограничения первичного ключа языка SQL определен первичный ключ.

Кроме этого, каждый кластеризованный индекс однозначен по умолчанию, т. е. в столбце, для которого определен кластеризованный индекс, каждое значение данных может встречаться только один раз. Если кластеризованный индекс создается для столбца, содержащего повторяющиеся значения, система баз данных принудительно обеспечивает однозначность, добавляя четырехбайтовый идентификатор к строкам, содержащим дубликаты значений.

Структура *не кластеризованного индекса* точно такая же, как и кластеризованного, но с двумя важными отличиями:

1. Не кластеризованный индекс не изменяет физическое упорядочивание строк таблицы.
2. Страницы листьев не кластеризованного индекса состоят из ключей индекса и закладок.

Если для таблицы определить один или более не кластеризованных индексов, физический порядок строк этой таблицы не будет изменен, таким образом, над таблицей их может быть создано множество. Для каждого не кластеризованного индекса СУБД создает дополнительную индексную структуру, которая сохраняется в индексных страницах.

1. Если существует кластеризованный индекс, то закладка не кластеризованного индекса показывает B+-дерево кластеризованного индекса таблицы.
2. Если таблица не имеет кластеризованного индекса, закладка идентична идентификатору строки (RID — Row Identifier), состоящего из трех частей:
 - a. адреса файла, в котором хранится таблица;
 - b. адреса физического блока (страницы), в котором хранится строка;
 - c. смещения строки в странице.

В обоих случаях количество операций ввода/вывода довольно велико, поэтому следует подходить к проектированию не кластеризованного индекса с осторожностью, и применять его только в том случае, если есть уверенность, что его использование существенно повысит производительность.

Влияние индексов на производительность СУБД

Наличие индекса во многих случаях повышает производительность операций выборки записей из таблицы. Для оценки производительности будем

производить выборку по одной записи из таблицы в цикле, без использования индексов и с использованием индексов.

Общий порядок определения влияния индексов на операции над таблицами следующий.

1. Создадим две идентичные таблицы вида.
2. Определим над одной из таблиц индекс. В нашем случае индекс с именем T2_F1_i определен над таблицей T2 и ее полем F1.
3. Заполним таблицу для начала десятью случайными значениями.

Пример кода для Postgre SQL:

```
-- Пример без индекса
CREATE TABLE T1(
    F1 int,
    F2 int
);
```

Обратите внимание на то, как кластеризованы данные в приведенном примере. Для получения случайных данных можно воспользоваться функциями СУБД. На Postgre SQL процедура генерации значений и выборки данных может выглядеть так:

```
create or replace function random_int(int) returns int as
$$
begin
    return random()*($1 - 1) + 1;
end;
$$ language plpgsql;

create or replace function insertRandomValues(int) returns void as
$$
declare
    i int := 0;
    maxValue int := 2048;
    first int := 0;
    second int := 0;
begin
    while i < $1 loop
        i := i + 1;
        first := (select * from random_int(maxValue));
        second := (select * from random_int(maxValue));
        insert into T1(F1, F2) values (first, second);
        insert into T2(F1, F2) values (first, second);
    end loop;
end;
$$ language plpgsql;
```

Важно, чтобы вносимые данные имели случайный характер для адекватного отображения поведения индекса.

4. Подготовим и произведем в автоматическом режиме на указанном массиве данных сто операций выборки, зафиксировав (также автоматически) время начала и окончания выполнения операций.

Для фиксации времени можно воспользоваться функциями GETDATE(), SYSDATETIME() или аналогов. Каждая современная СУБД имеет набор встроенных инструментов для получения текущих отметок времени разной точности. Ряд функций, включает смещение часового пояса, заданное в системе.

```
drop function if exists selectWithTimeT1(int, int);
create or replace function selectWithTimeT1(int, int) returns void as
$$
    declare
        startTime timestamp;
        endTime interval;
        count_of_records int;
    begin
        count_of_records := (select count(*) from T1);
        for i in 1..$2
            loop
                startTime := clock_timestamp();
                for i in 1 .. $1
                    loop
                        select f1 from T1 where f1 >= 64 and f1 <= 128;
                    end loop;
                endTime := clock_timestamp() - startTime;
                insert into time_table(try_number,
                    count_of_records, time_stamp)
                    values (i, count_of_records, endTime);
            end loop;
        end;
    $$ language plpgsql;
```

Измерение времени стоит производить несколько раз (рекомендуется шесть), для минимизации влияния случайных факторов на результат, и воспользоваться в итоге средним значением.

5. Повторить эксперимент для таблиц, в которых количество записей увеличивается от 10 до 1000000 с шагом в 10.

Помните, что перед вставкой большого количества записей следует сначала удалить индекс, а потом создать его заново – тогда процедура вставки будет работать быстрее. Пример графика для выборки данных из таблицы при наличии индекса и без него (при отсутствии кластеризации по атрибуту выборки) приведен на рисунке 4.1.

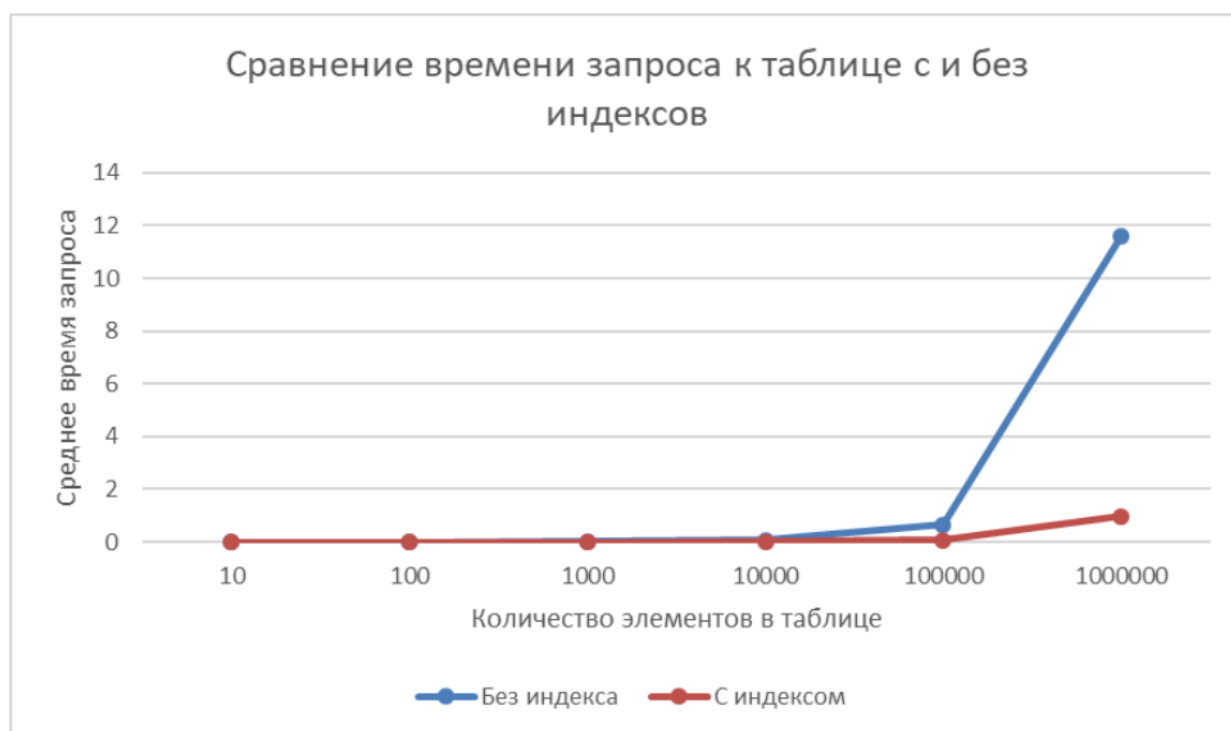


Рисунок 4.1. – Графики времени выполнения запроса данных при помощи индекса и последовательным чтением в зависимости от числа записей в таблице данных

Наличие индекса, как уже упоминалось, увеличивает время, затрачиваемое системой на вставку записей в таблицу. Для оценки производительности нам понадобится пять сходных таблиц:

1. без индекса;
2. с простым индексом;
3. с уникальным индексом;
4. с индексом по выражению;
5. с индексом с использованием пользовательской функции.

В каждую таблицу необходимо выполнить вставку 1000^3 записей с замером времени вставки (не менее шести раз в каждую, как и в предыдущем примере) с постепенным наращиванием исходного, до вставки, объема записей, уже находящихся в таблице. Значение в 1000 записей взято так как время вставки одной записи слишком мало для фиксации с учетом фоновых процессов. Обратите внимание, что после операции вставки необходимо очищать таблицу перед следующим замером, чтобы внесенные ранее данные и размер индекса не влияли

³ Значение 1000 может быть увеличено если временные интервалы получаются слишком маленькими для оценки с большим влиянием погрешностей.

на полученные результаты оценки времени. Примеры результатов замеров времени такой вставки для 100 записей приведены на рисунке 4.2.

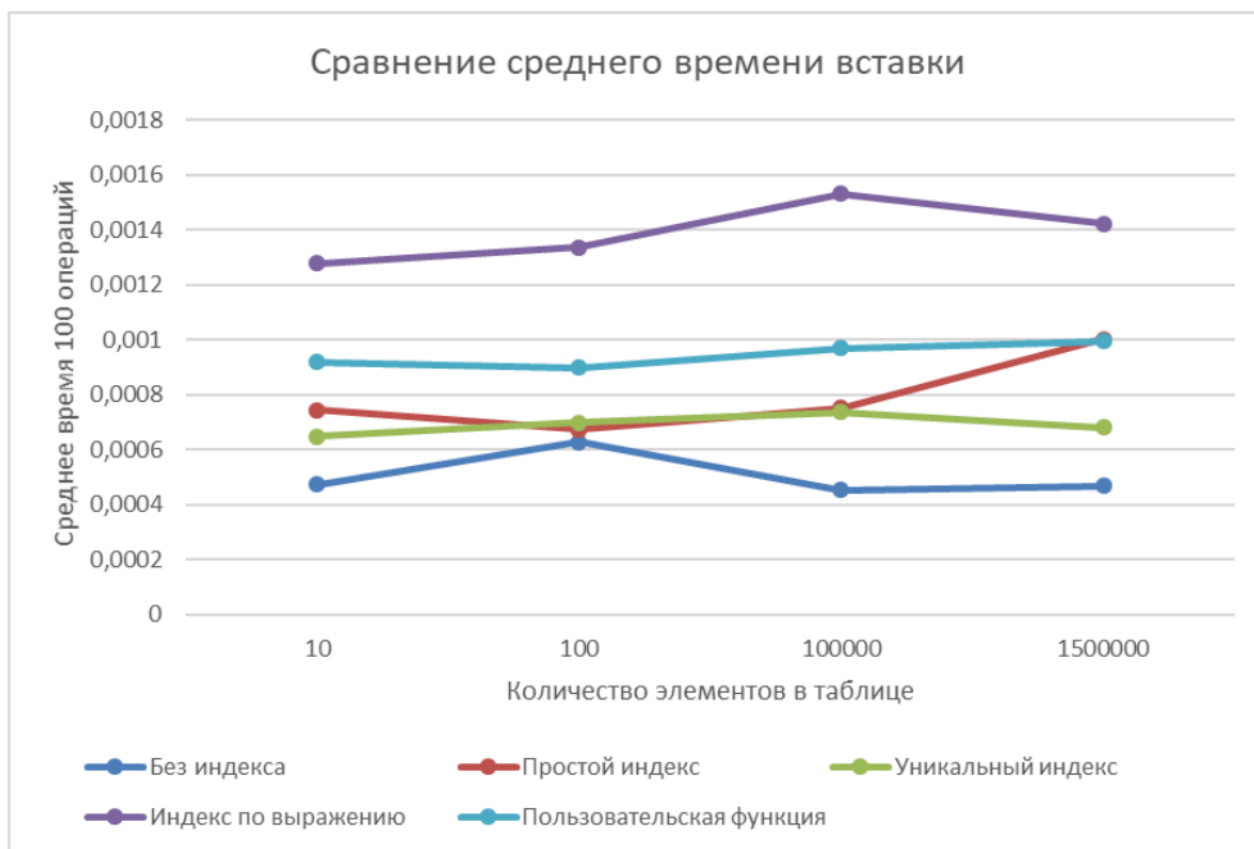


Рисунок 4.2. – Графики времени выполнения вставки данных при различных условиях индексации.

Наличие индекса увеличивает время, затрачиваемое системой на обновление записей в таблице с учетом типа самого индекса. Причем для индексов, состоящих из нескольких полей, коэффициент снижения производительности зависит от того, какое именно поле изменяется. Обратите внимание, что сложность вставки примерно одинакова, что обусловлено одним и тем же способом кластеризации самой таблицы в каждом случае, вне зависимости от наличия индекса.

Исследуйте зависимость снижения производительности обновления записей таблицы от типа поля, по которому происходит индексирование. Общий порядок исследования производительности обновления соответствует общему порядку исследования производительности вставки данных. Пример результатов обновления данных приведен на рисунке 4.3.

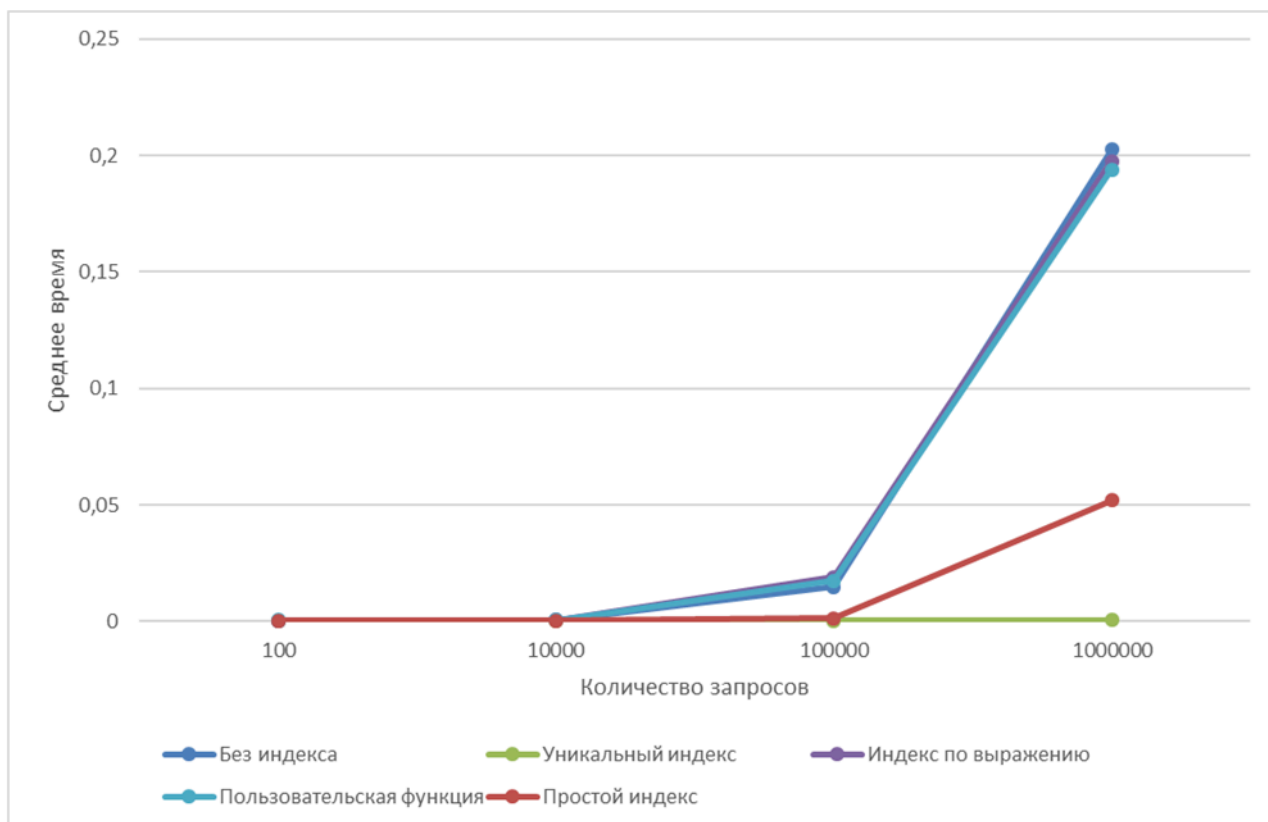


Рисунок 4.3. – Графики времени выполнения обновления данных при различных условиях индексации.

Обратите внимание, что время обновления данных (одной случайной записи) без индексации и с индексации на основе выражения и функции имеет одну сложность. То есть, при обновлении одной случайно записи индекс фактически не используется для ее поиска. Объясните этот эффект.

Методические указания по выполнению работы

Задание на работу

1. Определите размер файлов пользовательских и системных баз данных сервера СУБД.
2. Оцените рост хранилища данных на примере логгирования данных пользователя. Таблицу для логгирования используйте из работы 3, если она выполнялась, или задайте структуру самостоятельно. Для проведения эксперимента:
 - а. Определите для пустой таблицы, сколько места она занимает:
 - i. Воспользуйтесь запросом к системным таблицам сервера или предустановленным процедурам.
 - ii. Оцените размер физического хранения файла на диске
 - б. Добавьте в таблицу некоторое (от 10) количество записей.

- с. Повторите оценку пространства.
 - i. Если оно не изменилось, удвойте количество данных, пока разница не станет заметна.
 - ii. Последовательными экспериментами определите, насколько возможно точно, размер одной записи. Для этого используйте экспериментальное определение размера экстента (блока записи) и число записей в нем. Учтите особенность организации данных в первом экстенте.
 - d. Повторите проверку с дальнейшим увеличением числа записей (с приростом, не менее пяти - шести раз).
3. Определите, какой объем понадобится для хранения N записей. Обоснуйте результат.
4. Проверьте полученный результат, увеличив заполненное число записей в несколько раз (не менее пяти – шести проверок).
5. Проведите секционирование таблицы и его оценку. Для этого:
- a. Очистите таблицу.
 - b. Приведите таблицу в не кластеризованный вид, удалите индексы (при наличии).
 - c. Автоматически заполните таблицу значениями до размера в 1 000 000 записей. Данное количество может быть больше или меньше в зависимости от скорости оборудования. Если обработка записей происходит очень быстро (или очень медленно) их количество можно уменьшить (или, соответственно, увеличить) до получения таблицы с задержкой минимум в секундах, на операции простой выборки не кластеризованных данных.
 - d. Организуйте секционирование созданной таблицы средствами СУБД (если они доступны) и/или вручную.
 - e. Организуйте (при возможности) размещение секций на разных дисках.
6. Оцените скорость выполнения трех запросов* к секционированным данным:
- a. Для варианта размещения на одном диске (не менее 10 тестов)
 - b. Для варианта размещения на разных дисках (если он возможен, не менее 10 тестов)

* Для теста составьте три запроса одной селективности, для выполнения двух из которых нужны данные только одной из двух секций (соответственно

первой секции и второй секции для первого и второго запроса) а для третьего – обеих секций.

7. Проведите эксперимент по определению производительности выборки с использованием индексов, согласно приведенному в теоретической части плану:
 - a. Создайте тестовые отношения (без индекса и кластеризации, с индексом без кластеризации).
 - b. Разработайте функцию по выборки одного* случайного значения с замером времени операции.
 - c. Проведите тесты, постепенно увеличивая объем данных в тестовых отношениях используя случайные данные с низкой селективностью.
8. Постройте график зависимости времени выборки одной случайной записи от размера таблицы (то есть, общего числа записей в ней) для обоих случаев – без индекса и с индексом, и определите сложность операции выборки в каждом случае. Используйте один или два графика для представления обоих случаев, в зависимости от того, насколько они будут визуально показательны.
9. Аналогично проведите эксперимент по определению производительности вставки с использованием индексов, согласно приведенному плану и постройте графики.
 - d. Создайте тестовые отношения:
 - i. Без индекса и кластеризации
 - ii. Без кластеризации с простым индексом
 - iii. Без кластеризации с уникальным индексом
 - iv. Без кластеризации с индексом по выражению
 - v. Без кластеризации с индексом по функции
 - e. Разработайте функцию по вставки одного* случайного значения с замером времени операции.
 - f. Проведите тесты, постепенно увеличивая объем данных в тестовых отношениях используя случайные данные с низкой селективностью.
10. Постройте график – зависимость времени занесения одной случайной записи от размера таблицы для каждого из пяти индексов.
11. Аналогично эксперименту с операцией вставки, на тех же типах тестовых отношений, проведите эксперимент по определению производительности обновления данных и постройте графики.

* Если время выборки/обновления/вставки одной записи слишком мало, используйте 10, 100 или более случайных записей с последующим делением результата. Ответьте на вопрос, почему важен именно случайный характер данных при обращении.

Содержание отчета

1. Титульный лист.
2. Цель работы.
3. Задание на работу, включающее индивидуальное описание предметной области и СУБД.
4. Скрипт (с комментариями) и пример его выполнения по оценке размера файлов данных и журнала.
5. Таблица с размерами всех файлов журнала и данных баз данных сервера (если их несколько).
6. Данные по оценке пространства в табличной форме для 0, 10 и далее записей в табличной форме. Занести в таблицу все данные замеров, включая поиск времени выделения нового блока данных (значения до и после выделения блока).
7. Расчет и среднее значение объема для одной записи.
8. Расчет и среднее значение объема для N записей. Приведите результаты проверок в таблице, указав расчетное значение и полученное в результате измерения.
9. Описание и скрипт использования встроенных средств секционирования рассматриваемой СУБД (при наличии).
10. Схема данных после ручного секционирования таблицы. Скрипты, необходимые для секционирования и работы с разделенной таблицей.
11. Описание размещения секций на разных физических дисках (если возможно).
12. Временные оценки выполнения запросов. Привести для каждого запроса:
 - 12.1. SQL код запроса;.
 - 12.2. Таблицу времени выполнения (все тесты) с и без секционирования.
 - 12.3. Среднее значение времени выполнения (в обоих случаях).
13. Сводная таблица времени выполнения, где для каждого запроса приведено: [«SQL код», «Время без секционирования», «Время с

секционированием на одном диске»[, «Время с секционированием на разных дисках»]]

14. Скрипты по созданию таблиц и индексов для определения производительности операций (можно вынести в приложение 1).

15. Скрипты по заполнению таблиц случайными значениями с указанием, откуда взяты случайные данные, для определения производительности операций (можно вынести в приложение 2).

16. Перечень функций фиксации времени СУБД с особенностями их работы. Обоснование выбора функций фиксации времени.

17. Оценка производительности выборки данных

17.1. Скрипты выборки записей из таблиц с установкой временных интервалов операций для определения производительности выборки данных.

17.2. График зависимости времени выборки одной случайной записи от размера таблицы (для обоих случаев – без индекса и с индексом).

17.3. Выводы по зависимости в операции выборки данных.

18. Оценка производительности вставки данных

18.1. Скрипты по вставке данных в таблицы (и очистки таблиц) с установкой временных интервалов операций для определения производительности вставки данных.

18.2. График зависимости времени обновления одной записи от количества записей в наборе для каждого из пяти индексов.

18.3. Выводы по зависимости в операции вставки данных.

19. Оценка производительности обновления данных

19.1. Скрипты по обновлению данных в таблицах с установкой временных интервалов операций для определения производительности вставки данных.

19.2. График зависимости времени обновления одной записи от количества записей в наборе для каждого из пяти индексов.

19.3. Выводы по зависимости в операции обновления данных.

20. Заключение

21. Библиографический список

Варианты выполнения

Задание не имеет отдельных вариантов выполнения: каждый студент разрабатывает компоненты и код для экспериментов самостоятельно и проводит на своем оборудовании.

Контрольные вопросы

1. Как организуется физическое хранение данных в рассматриваемом сервере СУБД?
2. Какова структура первого блока данных? Последующих блоков?
3. Как секционирование влияет на производительность доступа к данным?
4. В каком случае секционирование улучшает характеристики доступа к данным?
5. В каком случае секционирование ухудшает характеристики доступа к данным?
6. Как изменятся результаты тестирования секционирования данных, если таблица будет кластеризована и/или будет иметь индекс, используемый при выполнении запроса?
7. Какой характер имеет зависимость времени выборки одной случайной записи от объема данных различных ситуациях?
8. Какую сложность имеет эта операция в каждом случае и почему?
9. Какой характер имеет зависимость времени вставки одной случайной записи от объема данных в различных ситуациях?
10. Какую сложность имеет эта операция в каждом случае и почему? Что влияет на время вставки данных в различных случаях?
11. Какой характер имеет зависимость времени обновления одной случайной записи от объема данных различных ситуациях?
12. Из каких операций складывается операция обновления данных?
13. Какую сложность имеет эта операция в каждом случае и почему? Почему характер временной зависимости для некоторых типов индексов отличается?
14. Что такое кластеризация данных в СУБД и как она влияет на скорости операций с данными?
15. Какой режим кластеризации надо установить при вставке данных? Как это сделать?
16. Какой режим кластеризации надо установить при поиске данных? Как это сделать?

17. Как кластеризовать таблицу данных по нескольким атрибутам?

Список источников

1. Дейт, К. Дж. Введение в системы баз данных, 8-е издание.: Пер. с англ. — М.: Издательский дом "Вильямс", 2005. — 1328 с
2. Кузнецов С. Д. Базы данных: учебник для студ. Учреждений высшего проф. Образования / С. Д. Кузнецов. — М.: Издательский центр «Академия», 2012. — 496с.
3. Моргунов Е. П. PostgreSQL. Основы языка SQL: учеб. пособие / Е. П. Моргунов; под ред. Е. В. Рогова, П. В. Лузанова. — СПб.: БХВ-Петербург, 2018. — 336 с.
4. Полтавцева М.А. Безопасность баз данных. Учебное пособие / М.А. Полтавцева. СПб. Изд-во Политехнического университета, 2023. – 142с.
5. Тарасов С.В. СУБД для программиста. Базы данных изнутри. – М.: СОЛОН-Пресс, 2015. – 320с.
6. Рогов Е. В. PostgreSQL 15 изнутри. — М.: ДМК Пресс, 2023. — 662 с.